

CLAUDE

CLAUDE.md

INHERITED FROM the Helix Constitution

This module is governed by the Helix Constitution. All rules in the constitution's CLAUDE.md and the Constitution.md it references apply unconditionally — the universal anti-bluff covenant §11.4, the no-guessing mandate §11.4.6, the credentials-handling mandate §11.4.10, host-session safety §12, data safety §9, and mutation-paired gates §1.1. Locate the constitution from any nested depth via its `find_constitution.sh` helper — do NOT hardcode a path (this module stays fully decoupled and project-agnostic per §11.4.28). The module-specific rules below extend the universal clauses; they never weaken any of them. When this file disagrees with the constitution, the constitution wins.

Canonical reference: <https://github.com/HelixDevelopment/HelixConstitution>

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

HelixQA — AI Agent Operating Manual

`digital.vasic.helixqa` is an **anti-bluff QA orchestration framework** for cross-platform testing. It runs YAML test banks across Android, Android TV, Web, and Desktop targets with real-time crash/ANR detection, step-by-step evidence collection, automated Markdown ticket generation, and an LLM-driven **Autonomous QA Session** mode that navigates applications, verifies documented features, discovers bugs, and records video evidence.

The design centre is the §11.4 Operative Rule: **the bar is not "tests pass" but "users can use the feature."** Every PASS HelixQA emits MUST carry positive runtime evidence captured during execution (screenshots, logcat, video, stack traces). A green summary line without that evidence is a critical defect of equal severity to a missing feature.

1. Agent Identity & Purpose

You are an AI agent working on **HelixQA**. Your mandate: write real, working, tested Go code. No simulations, no placeholders, no "for now" implementations. The framework's whole reason to exist is to catch bluffs — so a bluff landed in HelixQA itself defeats its purpose and is the most severe class of defect here.

2. Module Facts

- **Module ID:** `digital.vasic.helixqa` (single Go module; go 1.26).
- **CLI binary:** `bin/helixqa`, built from `cmd/helixqa`.
- **Subcommands:** `run`, `autonomous`, `http`, `replay`, `list`, `report`, `signoff`, `version`, `help` (see `cmd/helixqa/main.go`).
- **Test banks:** YAML/JSON documents under `banks/` (96 YAML + 66 JSON peers at last count) describing platform-targeted test cases.
- **License:** Apache-2.0.

2.1 Own-org dependencies

This module depends on sibling own-org Go modules consumed by reference (declared in `helix-deps.yaml`, never nested as `.gitmodules` entries per §11.4.28(C)): `digital.vasic.challenges`, `digital.vasic.containers`, `digital.vasic.security`, plus the autonomous-session integrations `LLMsVerifier`, `LLMOrchestrator`, `LLMProvider`, `VisionEngine`, and `DocProcessor`. `go.work` wires local replaces during development.

3. Build & Test Commands

```
go build ./...                # build every package
go build -o bin/helixqa ./cmd/helixqa # build the CLI (or: make build)
go test ./... -count=1        # all tests, cache disabled (or: make test)
go vet ./...                  # static analysis (or: make vet)

# Single package / single test
go test -v ./pkg/testbank/ -count=1
go test -v -run TestSomething ./pkg/orchestrator/

make test-race                # tests with -race
make test-cover               # coverage report → coverage.html
make lint                     # golangci-lint run ./...
make fmt                      # gofmt -w .
```

3.1 Running QA pipelines and banks

```
# Run the QA pipeline across platforms
bin/helixqa run --banks banks/ --platform all
```

```

# Android-specific run against a device/emulator
bin/helixqa run --banks banks/ --platform android \
  --device emulator-5554 --package com.example.app

# List test cases discovered in banks
bin/helixqa list --banks banks/ --platform android

# Generate a report from existing results
bin/helixqa report --input qa-results --format html

# Autonomous LLM-driven QA session against a project
bin/helixqa autonomous --project /path/to/your-project \
  --platforms desktop --env .env --timeout 30m --output qa-
  results/

```

3.2 Anti-bluff gates and challenges

```

make anti-bluff          # CONST-035 gates: scanner + behaviour-
                        anchor
                        # manifest + mutation ratchet (changed
                        files)
make anti-bluff-scan     # static bluff scanner over the full
                        tree
make anti-bluff-anchors  # behaviour-anchor manifest validator
make challenge           # run every challenges/scripts/*.sh
make qa-all             # challenge scripts + all anti-bluff
                        gates

```

Note (per the Makefile): `make qa-all` deliberately excludes `vet/test` because some packages require `own-org` replace directives that may be absent in a clean checkout; run `go test ./...` separately once those siblings are wired via `go.work`.

4. Architecture

The CLI in `cmd/helixqa/` dispatches subcommands to the orchestrator. The domain code lives under `pkg/` (~50 packages). The packages most central to the framework:

- **pkg/config** — configuration types and validation (`.env`-driven).
- **pkg/testbank** — YAML/JSON test-bank loading with platform/priority filtering.
- **pkg/detector** — platform-specific crash/ANR detection: `android.go` (ADB: `pidof`, `logcat`, `screencap`), `web.go` (browser process monitoring), `desktop.go` (JVM/process monitoring).
- **pkg/validator** / **pkg/validators** — step-by-step validation with evidence at each step to prevent false positives.
- **pkg/evidence** — centralized evidence collection (screenshots, video, logs, stack traces).
- **pkg/ticket** — Markdown ticket generation for AI fix pipelines.
- **pkg/reporter** — QA report generation (Markdown/HTML/JSON).
- **pkg/orchestrator** — the main QA pipeline coordinator.

- **pkg/autonomous** — SessionCoordinator, PlatformWorker, PhaseManager for the 4-phase autonomous session (Setup → Doc-Driven Verification → Curiosity-Driven Exploration → Report).
- **pkg/navigator** / **pkg/visionnav** — navigation engine and action executors (ADB, Playwright, X11) plus vision-driven navigation.
- **pkg/issuedetector** — LLM-powered bug detection (visual, UX, accessibility, functional).
- **pkg/session** / **pkg/recordingqa** — session recording, timeline, video management, and recording-quality validation.
- **pkg/bridge** / **pkg/bridges** / **pkg/capture** — device bridges and frame capture (scrcpy/x11grab/kmsggrab and related capture backends).
- **pkg/vision** / **pkg/gpu** — vision analysis and GPU-accelerated comparison (LPIPS, DreamSim — see the cmd/helixqa-* tools).
- **pkg/planning** — test-plan generation, including the generic Android TV Channels testing framework.
- **pkg/llm** — LLM client wiring for issue detection and question generation.

The auxiliary cmd/helixqa-* and cmd/ocu-* binaries are capture/vision helper tools (bridge, capture, axtree, omniparser, uitars, lpips, dreamsim, recording-analyzer, etc.) invoked by the framework.

See [ARCHITECTURE.md](#) and [API_REFERENCE.md](#) for full detail.

5. Test-bank conventions

Banks under banks/ are YAML documents (with optional JSON peers sharing the base name so tooling can auto-pair). Required top-level keys: version (string), name, test_cases (array). Per-test-case keys: id (TC-XXX), name, category (functional / performance / security / ux / chaos / ...), priority (critical / high / medium / low), platforms (subset of [android, android_tv, web, desktop, ios, aurora_os, harmony_os]), steps[] (each name / action / expected), tags[], and optional documentation_refs[] for traceability into docs/. Banks describe **structure**, not prose — name / expected strings drive LLM-generated prompts at runtime, so avoid hardcoded user-facing English in banks.

6. Anti-bluff principles (universal — apply to all code here)

Before marking any task complete, verify:

- **No simulation** — code does not contain "simulate", "for now", "TODO implement", "placeholder". Run the bluff scanner (make anti-bluff-scan).
- **Real I/O** — HTTP clients make real requests; device detection runs real ADB/process calls; file ops use real os calls; command execution uses os/exec and surfaces real exit codes — never Printf + Sleep.
- **Mocks are unit-test-only** — integration / e2e / autonomous / Challenge paths exercise the real system (§11.4.27).
- **Tests validate reality** — assert observable behaviour and captured evidence, not just call counts.
- **Every gate has a paired mutation** — a new gate ships with a §1.1 mutation proving it catches the regression it claims to.

- **No bare skips** — every `t.Skip()` is topology-justified with a reason (§11.4.3); PASS-by-default is forbidden.
- **Evidence captured** — a PASS for a user-visible behaviour cites the artifact (screenshot/video/logcat/report) proving it.

7. Conventions

- **Go**: constructor injection (`NewXxx(cfg)`), table-driven tests, `*_test.go` beside source. Wrap errors with `fmt.Errorf + %w`. Respect `context.Context` cancellation in long-running scans and sessions.
- **Concurrency**: platform workers and capture loops run concurrently; bound parallelism and reap child processes/goroutines on cleanup.
- **Configuration**: all runtime settings load from `.env` (copy `.env.example`). Credentials are never committed (§11.4.10).
- **Static analysis**: `go vet ./...` must be clean before commit.

8. Peer governance documents (keep in sync)

This `CLAUDE.md` sits alongside `AGENTS.md`, `QWEN.md`, and `CONSTITUTION.md` in this module. They share the same inheritance pointer and anti-bluff posture; cascade any rule change across all of them so an agent reading any one file gets the same binding rules.